

RECURSO COMPLEMENTARIO

Fundamentos de Transformers

Nota técnica 02

Tokens, vocabulario y batches

Curso abierto

2026-I

Tokenizar es elegir el alfabeto discreto con el que un modelo podrá observar, contar, aprender y predecir.

1 Discretización del texto

Un Transformer no recibe texto crudo. Recibe secuencias de enteros. Para llegar a esos enteros se define primero una segmentación del texto en unidades discretas: caracteres, palabras, subpalabras u otra unidad construida por un algoritmo de tokenización. Esa elección determina qué eventos existen para el modelo.

Sea un texto transformado en una secuencia de tokens

$$w_{1:N} = (w_1, w_2, \dots, w_N), \quad w_i \in \mathcal{V}.$$

Aquí $\mathcal{V}$ es el vocabulario: el conjunto finito de tokens que el sistema reconoce. El vocabulario no es un detalle de almacenamiento; define el soporte de la distribución de salida. Si un token no pertenece a $\mathcal{V}$, el modelo no puede predecirlo como unidad.

Idea clave 1 (Representación discreta). La tokenización convierte un objeto lingüístico continuo en una secuencia discreta. Esa conversión simplifica el problema computacional, pero también impone una pérdida: el modelo solo razona sobre las unidades que la tokenización le entrega.

Ampliación conceptual

La tokenización es una decisión de modelado porque determina qué eventos discretos serán contables. Si se trabaja a nivel de caracteres, el vocabulario es pequeño y la cobertura es alta, pero las dependencias útiles pueden requerir secuencias más largas. Si se trabaja a nivel de palabras, las unidades son más interpretables, pero aparecen problemas de vocabulario abierto. Las subpalabras buscan un compromiso: permiten representar palabras no vistas combinando piezas frecuentes.

La codificación como índice entero es deliberadamente pobre desde el punto de vista semántico. Su ventaja es que permite almacenar y manipular secuencias de forma eficiente. Su desventaja es que, antes de los embeddings, no hay noción de similitud: dos índices consecutivos pueden corresponder a tokens sin relación lingüística, y tokens relacionados pueden quedar lejos en la numeración.

La construcción de ventanas X/Y convierte un texto largo en muchas tareas pequeñas de predicción. Cada fila de X contiene contexto; cada fila de Y contiene los objetivos desplazados. Esta alineación es la primera forma concreta de la factorización autoregresiva: para cada columna temporal se entrena una predicción del siguiente token.

2 Codificación como función

Una vez definido el vocabulario, se construye una función de codificación

$$\text{id} : \mathcal{V} \rightarrow \{0, 1, \dots, V - 1\}, \quad V = |\mathcal{V}|.$$

Esta ecuación se lee así: cada token reconocido recibe una llave entera única entre cero y $V - 1$. El índice no mide frecuencia, importancia ni cercanía semántica; solo identifica una fila futura en una tabla.

La decodificación parcial se expresa como

$$\text{tok} : \{0, 1, \dots, V - 1\} \rightarrow \mathcal{V}.$$

Si el tokenizador es reversible para el dominio usado, aplicar codificación y decodificación conserva la secuencia de tokens. Esa reversibilidad no implica conservar todos los matices del texto original: normalización, mayúsculas, puntuación o espacios pueden haberse modificado antes.

3 Ventanas autoregresivas

Dada una secuencia codificada $x_{1:N}$, el entrenamiento autoregresivo no usa la secuencia completa como una sola muestra. Construye ventanas de longitud fija. Para una posición inicial s y longitud de contexto T , se define

$$\begin{aligned} X_s &= (x_s, x_{s+1}, \dots, x_{s+T-1}), \\ Y_s &= (x_{s+1}, x_{s+2}, \dots, x_{s+T}). \end{aligned}$$

Esta ecuación se lee así: Y_s es la misma ventana desplazada una posición hacia adelante. Cada elemento de X_s sirve como contexto para predecir el elemento alineado en Y_s .

Si la secuencia tiene longitud N , el número máximo de ventanas contiguas de longitud T con objetivo desplazado es

$$N_{\text{ventanas}} = N - T.$$

Aumentar T da más contexto por ejemplo, pero reduce la cantidad de ventanas disponibles en una secuencia fija. Esa tensión aparece incluso antes de construir la arquitectura.

4 Lectura tensorial

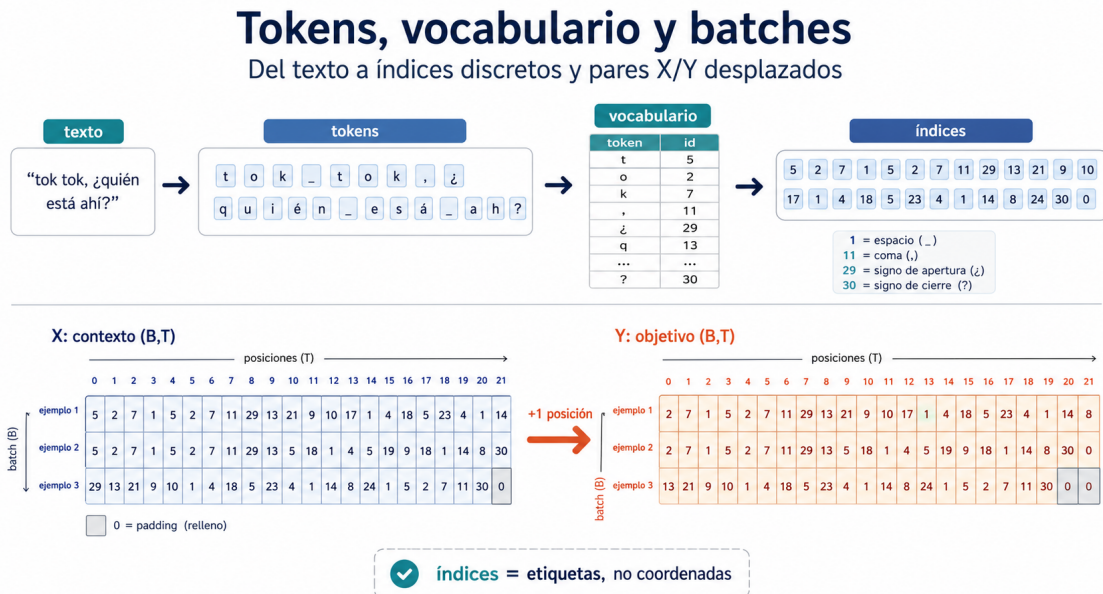
Un batch de ventanas se representa como

$$X \in \mathbb{N}^{B \times T}, \quad Y \in \mathbb{N}^{B \times T}.$$

La dimensión B agrupa ejemplos; T recorre posiciones; cada entrada es un índice entero. Todavía no hay vectores densos ni canales continuos. La forma (B, T) describe solo una matriz de llaves discretas.

Más adelante, una capa de embeddings transformará esos índices en tensores (B, T, C) . La diferencia es fundamental: (B, T) contiene identificadores; (B, T, C) contiene representaciones entrenables.

5 Figura conceptual



La figura resume la cadena técnica: texto normalizado, tokens, vocabulario, índices y ventanas X/Y. El punto crítico es el desplazamiento: el modelo no aprende a copiar X , sino a asignar probabilidad al siguiente índice de Y .

Cómo leer la figura

La figura debe leerse de izquierda a derecha como una pérdida gradual de forma textual y una ganancia de forma computacional. El texto conserva rasgos visuales y lingüísticos; los tokens son unidades discretas; el vocabulario fija el conjunto permitido; los índices permiten cómputo; las ventanas organizan ejemplos de entrenamiento.

La parte más importante es el desplazamiento entre entrada y objetivo. No se trata de duplicar datos, sino de construir pares supervisados: dado este prefijo, el siguiente índice observado fue este otro. Esa relación será usada por la pérdida de entropía cruzada.

6 Límites de interpretación

Un vocabulario pequeño hace visible la mecánica, pero restringe lo que puede expresarse. Un vocabulario grande mejora cobertura, pero aumenta la dimensión de salida V , el costo de la softmax y la cantidad de parámetros asociados. Un contexto corto limita dependencias largas; un contexto largo requiere más memoria y más cómputo.

Además, el tamaño del vocabulario afecta directamente la dificultad de la predicción. Una salida sobre $V = 50$ tokens no tiene el mismo costo ni la misma ambigüedad que una salida sobre decenas de miles de tokens. Por eso las decisiones de tokenización se conectan con memoria, velocidad y calidad de entrenamiento.

Observación 1 (Índices). El hecho de que un token tenga índice mayor que otro no significa que sea más grande, más importante ni más cercano semánticamente. El orden de índices es una convención de codificación.

7 Resumen

Resumen 1 (Cierre de la nota). Tokenizar define el vocabulario observable, codificar asigna llaves enteras y construir ventanas X/Y convierte una secuencia en ejemplos autoregresivos. Esta preparación fija el soporte de salida, la longitud de contexto y la forma inicial de los tensores.

8 Preguntas de estudio

1. ¿Por qué el vocabulario define el soporte de la distribución de salida?
2. ¿Qué información se pierde al normalizar o segmentar texto antes de tokenizar?
3. ¿Por qué Y debe ser X desplazado una posición?
4. ¿Qué cambia al aumentar T en una secuencia de longitud fija?
5. ¿Por qué un índice entero no debe interpretarse como una magnitud lingüística?

9 Continuidad

Una vez contruidos índices y ventanas, el siguiente paso es reemplazar cada llave entera por una fila entrenable: la tabla de embeddings.